Segmentation

Week 8 — Splitting an image into regions

Practical Image Processing

Lecture 8 ▪ 60–70 min

Recap & today

- Wk2/7: thresholding + morphology give clean binary shapes.
- Today: **segmentation** — partitioning an image into meaningful regions (object vs background, or each object separately).

DocScan milestone

Segment each digit as its own region, even when two digits touch — so they can be read one at a time.

Roadmap

Threshold-based segmentation

Region-based segmentation

Watershed: separating touching objects

Connected components

In-class exercise

Deep dive

Wrap-up

Threshold-based segmentation

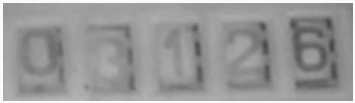
Segmentation = assign each pixel a label

- The goal: group pixels into regions that mean something (a digit, the page, the sky).
- Simplest case: two labels (object / background) = **thresholding** (Week 2).
- Harder: many objects, touching objects, or regions defined by color/texture.

Key idea

Segmentation is where "pixels" become "objects." Everything downstream (counting, reading) depends on it.

Global vs adaptive threshold — on the meter



gray



global Otsu



adaptive

Otsu picks one global threshold (great for even lighting); adaptive computes a local one (robust to shadows). Recall Week 2 — this is our starting point.

Otsu, revisited

- Finds the threshold that best separates the two histogram modes (max between-class variance).
- Works when the histogram is clearly **bimodal** (object vs background).
- Fails on: uneven lighting, more than two classes, or a unimodal histogram.

Math

Maximize $\sigma_B^2(t) = w_0(t)\, w_1(t)\, [\mu_0(t) - \mu_1(t)]^2$ over the threshold t.

Region-based segmentation

Beyond thresholds

- **Region growing**: start from seeds, add neighbours that are similar enough.
- **Split-and-merge**: divide the image, then merge regions that are alike.
- **Color/k-means**: cluster pixels by color into k regions.

Key idea

Thresholding uses only intensity; region methods use *similarity and connectivity* — more powerful, more parameters.

k-means colour segmentation — real car



original



k-means, k=4 colours

- Each pixel is a point in (R,G,B); k-means groups them into k clusters.

k-means: how it works

1. Pick k initial cluster centres.
2. Assign each pixel to the nearest centre.
3. Recompute each centre as the mean of its pixels.
4. Repeat 2–3 until stable.

✕ Pitfall

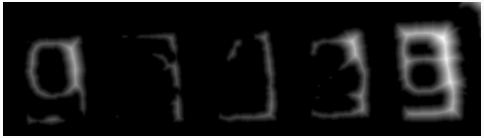
You must choose k; the wrong k over- or under-segments. It also ignores *position* — only colour.

Watershed: separating touching objects

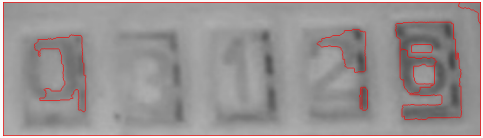
The touching-objects problem

- Threshold + morphology gives blobs — but two touching digits become *one* blob.
- Counting/reading needs them separated. Erosion helps a little, but can break thin strokes.
- **Watershed** solves it properly, using the shape (a distance transform).

Distance transform → watershed



distance transform



watershed boundaries (red)

The distance transform peaks at each object's centre; watershed "floods" from those peaks and builds a ridge where floods meet — the separating boundary.

Watershed intuition

- Treat intensity (or −distance) as a landscape; "water" rises from marked minima.
- Where two catchment basins meet, build a dam — that dam is the object boundary.
- **Markers** (seeds) control it, or it **over-segments** (every tiny minimum a region).

✓ Good practice

Marker-controlled watershed is the reliable version: you tell it the sure-foreground and sure-background.

Watershed in code (marker-controlled)

```
dist = cv2.distanceTransform(binary, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.5*dist.max(), 255, 0)  # digit centres
sure_bg = cv2.dilate(binary, k, iterations=3)
unknown = cv2.subtract(sure_bg, np.uint8(sure_fg))
_, markers = cv2.connectedComponents(np.uint8(sure_fg))
markers = markers + 1; markers[unknown==255] = 0
markers = cv2.watershed(color_img, markers)               # -1 = boundaries
```

Connected components

Labelling separate blobs



each blob a different colour

- **Connected components** labels each separate white region with an integer.
- Once separated, each digit is its own component you can crop and read (Week 9).
- `cv2.connectedComponents` / `...WithStats` give labels + areas + boxes.

In-class exercise

In-class exercise (12 min)

Choose a method

Which segmentation method fits each task?

1. Separate dark text from a light page under even lighting.
2. Two meter digits are touching and must be counted separately.
3. Segment a photo into “sky / road / car” regions by colour.
4. A document photographed with a shadow across half of it.

Debrief

- (1) Even lighting, bimodal → **Otsu**. (2) Touching objects → **watershed**.
- (3) Colour regions → **k-means** (or colour thresholding in HSV).
- (4) Shadow → **adaptive** threshold (local), not global Otsu.

Key idea

No single method wins. Match the method to the structure: intensity, colour, connectivity, or lighting.

Deep dive

Evaluating a segmentation

- With ground-truth masks: **IoU** (intersection over union) and **Dice** score.
- Over-segmentation (too many regions) vs under-segmentation (merged regions) — both are errors.
- For DocScan: the real test is downstream — can each digit be read after segmentation?

Math

$\mathrm{IoU} = \dfrac{|A \cap B|}{|A \cup B|}$ — 1.0 is perfect overlap with the truth.

Classical vs deep segmentation

- Classical (threshold/watershed/k-means): fast, interpretable, no training — great for clean, structured images.
- Deep (U-Net, Mask R-CNN, SAM): learns to segment complex scenes, but needs data/compute.
- For meters/documents, classical often suffices; deep shines on messy natural scenes (Week 13–14).

GrabCut & graph-based methods (brief)

- **GrabCut**: model foreground/background colour distributions and cut the graph optimally — interactive object cutout.
- Graph-cut/normalized-cut view segmentation as partitioning a pixel graph.
- These bridge classical and modern; good to know they exist.

The segmentation pipeline for DocScan



- Threshold → clean → separate touching digits → label each → read.
- Each digit becomes an isolated image the CNN can classify.

Wrap-up

What you should now be able to do

- Segment by threshold (global/adaptive/Otsu) and know when each fails.
- Use k-means for colour segmentation; region methods for similarity.
- Separate touching objects with a marker-controlled watershed.
- Label blobs with connected components; evaluate with IoU/Dice.

This week's deliverable

Due before Week 9 (see `project/`)

1. Compare Otsu vs adaptive on meter images; pick the winner per lighting.
2. k-means colour segmentation of a car/plate photo at $k = 2, 4, 8$; discuss.
3. Watershed to separate touching meter digits; label each with connected components.
4. DocScan milestone: output one cropped image per digit, in order.

Segmentation turns pixels into objects.

Threshold for two classes, k-means for colour, watershed for touching objects.